

TASKS:

PART 1: FILE INFORMATION GATHERING

Create a folder named:

Forensics_Lab

Download or create the following files:

- **document.pdf**
- **image.jpg**
- **sample.txt**
- **archive.zip**

Tasks:

- 1. Identify:**
 - **File Name**
 - **File Extension**
 - **File Size**
 - **File Type**
 - **Last Modified Date**
- 2. Use Linux commands:**
 - **ls -lh**
 - **file**
 - **stat**

Take screenshots.

PART 2: HASH ANALYSIS

Generate hashes for all files.

Use:

- **md5sum**
- **sha256sum**

Tasks:

- 1. Generate MD5 hash.**
- 2. Generate SHA256 hash.**
- 3. Compare hashes of two identical files.**

4. Modify a file and generate the hash again.

Answer:

- Did the hash change?
- Why?

PART 3: METADATA ANALYSIS

Install and use ExifTool.

Analyze:

- image.jpg
- document.pdf

Find:

- Creation Date
- Author
- Camera Information (if available)
- Software Used
- GPS Information (if available)

Take screenshots.

PART 4: INTEGRITY VERIFICATION

Tasks:

1. Create a file named evidence.txt.
2. Generate its hash.
3. Modify the file.
4. Generate the hash again.

Explain:

- Why digital evidence integrity is important.
- How hashing helps investigators.

PART 5: INVESTIGATION REPORT

Prepare a report containing:

1. File Details

- 2. Hash Values**
- 3. Metadata Analysis**
- 4. Screenshots**
- 5. Observations**
- 6. Security Recommendations**

Analyze 5 files from your computer and determine:

- File Type**
- Hash Value**
- Metadata Available**
- Potential Security Risks**

SUBMISSION

- PDF or Word Document**

PART 1: FILE INFORMATION GATHERING

Objective

The objective of this task was to gather basic information about different files using Linux forensic commands. The analysis included identifying the file name, extension, size, type, and last modified date of each file.

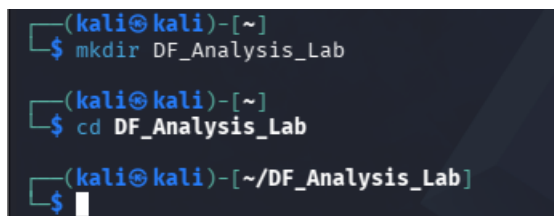
Step 1: Create a Working Directory

A dedicated directory named **DF_Analysis_Lab** was created to organize and store all files required for the forensic investigation.

Command Used

```
mkdir DF_Analysis_Lab  
cd DF_Analysis_Lab
```

Screenshot



```
(kali㉿kali)-[~]  
$ mkdir DF_Analysis_Lab  
  
(kali㉿kali)-[~]  
$ cd DF_Analysis_Lab  
  
(kali㉿kali)-[~/DF_Analysis_Lab]  
$
```

Step 2: Create and Collect Required Files

After creating the folder, the required files were created and stored inside the directory for forensic analysis.

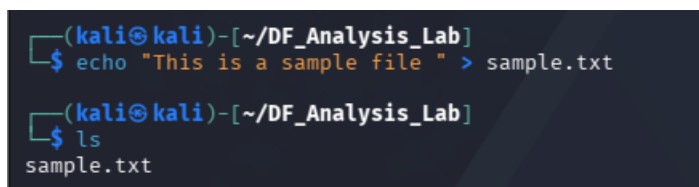
Step 2.1: Create sample.txt

A text file named **sample.txt** was created using the Linux terminal.

Command Used:

```
echo "This is a sample text file for forensic analysis." > sample.txt
```

Screenshot :



```
(kali㉿kali)-[~/DF_Analysis_Lab]  
$ echo "This is a sample file " > sample.txt  
  
(kali㉿kali)-[~/DF_Analysis_Lab]  
$ ls  
sample.txt
```

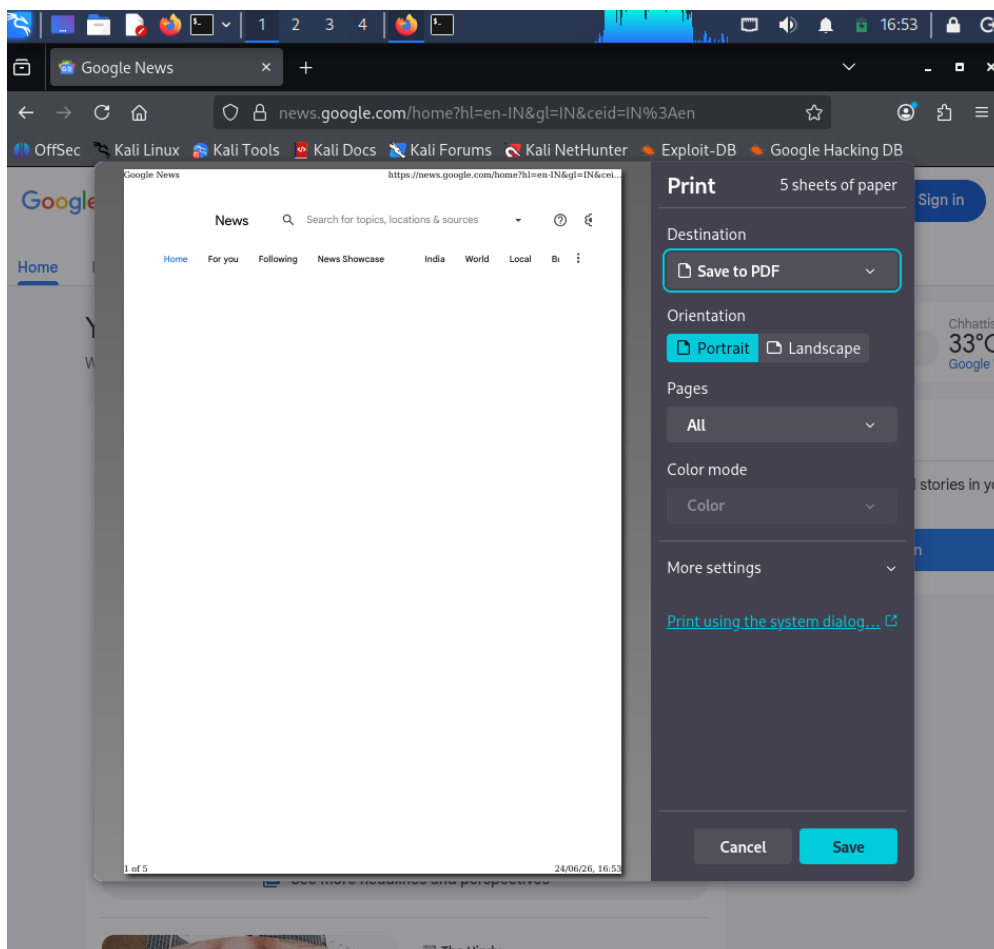
Step 2.2: Create document.pdf

A PDF document named **document.pdf** was created using Firefox's Print to PDF feature.

Procedure:

1. Open Firefox browser.
2. Open any webpage.
3. Press **Ctrl + P**.
4. Select **Print to File**.
5. Choose **PDF** format.
6. Save the file as **document.pdf** inside the **DF_Analysis_Lab** folder.

Screenshot :



```
(kali@kali)-[~/DF_Analysis_Lab]
$ ls
document.pdf  sample.txt
```

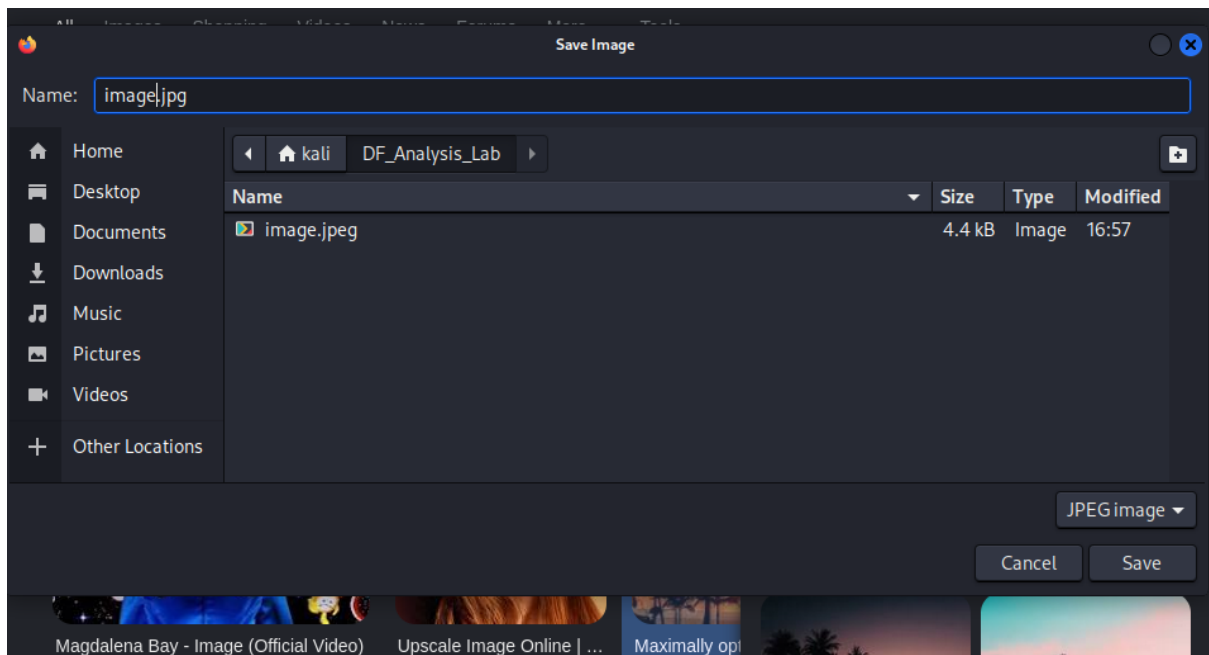
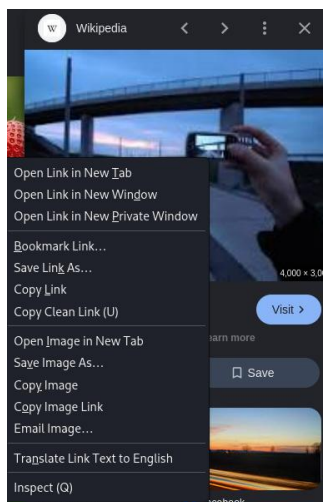
Step 2.3: Create image.jpg

A JPEG image file named **image.jpg** was obtained and saved inside the **DF_Analysis_Lab** folder.

Procedure:

1. Open any image in a web browser.
2. Right-click on the image.
3. Select **Save Image As**.
4. Save the image as **image.jpg** inside the **DF_Analysis_Lab** folder.

Screenshot :



Step 2.4: Create archive.zip

A ZIP archive named **archive.zip** was created by compressing the sample.txt file.

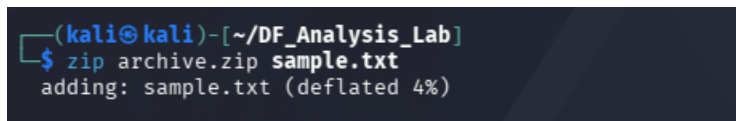
Command Used:

```
zip archive.zip sample.txt
```

Purpose:

The ZIP file was used to analyze compressed file formats.

Screenshot :



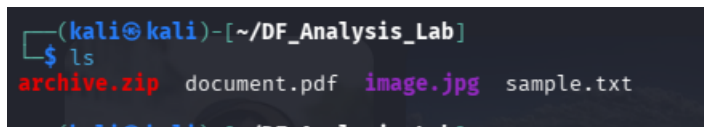
```
(kali㉿kali)-[~/DF_Analysis_Lab]
$ zip archive.zip sample.txt
adding: sample.txt (deflated 4%)
```

Step 2.5: Verify All Files

The contents of the **DF_Analysis_Lab** folder were verified to ensure that all required files were available.

Command Used:ls

Screenshot :



```
(kali㉿kali)-[~/DF_Analysis_Lab]
$ ls
archive.zip  document.pdf  image.jpg  sample.txt
```

Step 3: Display File Information Using ls -lh

The ls -lh command was used to display all files present in the **DF_Analysis_Lab** directory along with their file sizes in a human-readable format. This command helps investigators quickly identify the files available for analysis and determine their sizes.

Command Used

```
ls -lh
```

Explanation

- **ls** → Lists files and directories.
- **-l** → Displays detailed information in long format.
- **-h** → Shows file sizes in a human-readable format (KB, MB, etc.).

Screenshot :

```
(kali㉿kali)-[~/DF_Analysis_Lab]
$ ls -lh
total 2.7M
-rw-rw-r-- 1 kali kali 192 Jun 24 16:58 archive.zip
-rw-rw-r-- 1 kali kali 2.3M Jun 24 16:54 document.pdf
-rw-rw-r-- 1 kali kali 405K Jun 24 17:04 image.jpg
-rw-rw-r-- 1 kali kali 23 Jun 24 16:51 sample.txt
```

Step 4: Identify File Types Using the file Command

The file command was used to determine the actual type of each file. Unlike file extensions, which can be changed manually, the file command examines the file contents and identifies the true file format.

Commands Used

file document.pdf

file image.jpg

file sample.txt

file archive.zip

Explanation

The file command analyzes the internal structure of a file and reports its actual format.

- **document.pdf** → Identified as a PDF document.
- **image.jpg** → Identified as a JPEG image file.
- **sample.txt** → Identified as an ASCII text file.
- **archive.zip** → Identified as a ZIP archive.

This command is useful in digital forensics because attackers may rename malicious files with misleading extensions. The file command helps verify the true nature of a file.

Screenshot :

```
(kali㉿kali)-[~/DF_Analysis_Lab]
$ file document.pdf
document.pdf: PDF document, version 1.7

(kali㉿kali)-[~/DF_Analysis_Lab]
$ file image.jpg
image.jpg: JPEG image data, baseline, precision 8, 1920x1080, components 3

(kali㉿kali)-[~/DF_Analysis_Lab]
$ file sample.txt
sample.txt: ASCII text

(kali㉿kali)-[~/DF_Analysis_Lab]
$ file archive.zip
archive.zip: Zip archive data, made by v3.0 UNIX, extract using at least v2.0, last modified Jun 24 2026 16:51:12,
uncompressed size 23, method=deflate
```

Step 5: Obtain Detailed File Information Using the stat Command

The stat command was used to obtain detailed information about each file present in the **DF_Analysis_Lab** directory. This command provides important metadata such as file size, permissions, ownership, and timestamps, which are useful during forensic investigations.

Commands Used

stat document.pdf

stat image.jpg

stat sample.txt

stat archive.zip

Explanation

The stat command provides the following information:

- **File Name** – Name of the file being analyzed.
- **Size** – Size of the file in bytes.
- **File Type** – Indicates whether the file is a regular file, directory, etc.
- **Permissions** – Read, write, and execute permissions assigned to the file.
- **Owner and Group** – User and group associated with the file.
- **Access Time** – Last time the file was accessed.
- **Modify Time** – Last time the file content was modified.
- **Change Time** – Last time file metadata was changed.
- **Birth Time** – File creation time (if supported by the filesystem).

Screenshot :

```
(kali㉿kali)-[~/DF_Analysis_Lab]
$ stat document.pdf
File: document.pdf
Size: 2319765      Blocks: 4536      IO Block: 4096   regular file
Device: 8,1      Inode: 403745      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   kali)   Gid: ( 1000/   kali)
Access: 2026-06-24 17:00:11.449096811 +0530
Modify: 2026-06-24 16:54:40.170700100 +0530
Change: 2026-06-24 16:54:40.176773420 +0530
Birth: 2026-06-24 16:54:40.170700100 +0530
```

```
(kali㉿kali)-[~/DF_Analysis_Lab]
$ stat image.jpg
File: image.jpg
Size: 414164      Blocks: 816      IO Block: 4096   regular file
Device: 8,1      Inode: 405151      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   kali)   Gid: ( 1000/   kali)
Access: 2026-06-24 17:07:13.463355078 +0530
Modify: 2026-06-24 17:04:31.929875931 +0530
Change: 2026-06-24 17:04:31.953875581 +0530
Birth: 2026-06-24 17:04:31.929875931 +0530
```

```
(kali㉿kali)-[~/DF_Analysis_Lab]
$ stat sample.txt
File: sample.txt
Size: 23          Blocks: 8          IO Block: 4096   regular file
Device: 8,1      Inode: 400441      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   kali)   Gid: ( 1000/   kali)
Access: 2026-06-24 16:58:06.710113640 +0530
Modify: 2026-06-24 16:51:11.101302926 +0530
Change: 2026-06-24 16:51:11.101302926 +0530
Birth: 2026-06-24 16:51:11.101302926 +0530

(kali㉿kali)-[~/DF_Analysis_Lab]
$ stat archive.zip
File: archive.zip
Size: 192         Blocks: 8          IO Block: 4096   regular file
Device: 8,1      Inode: 405150      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   kali)   Gid: ( 1000/   kali)
Access: 2026-06-24 17:00:39.908809893 +0530
Modify: 2026-06-24 16:58:06.710113640 +0530
Change: 2026-06-24 16:58:06.710113640 +0530
Birth: 2026-06-24 16:58:06.692409352 +0530
```

PART 2: HASH ANALYSIS

Objective

The objective of this task was to generate and analyze cryptographic hash values for different files. Hash values provide a unique digital fingerprint for a file and are commonly used in digital forensics to verify file integrity and detect unauthorized modifications.

Step 1: Generate MD5 Hash Values

The MD5 hashing algorithm was used to generate hash values for all files .

Commands Used

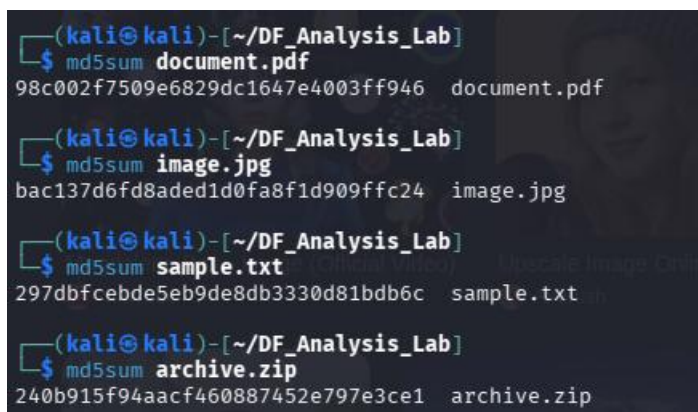
```
md5sum document.pdf
```

```
md5sum image.jpg
```

```
md5sum sample.txt
```

```
md5sum archive.zip
```

Screenshot



```
(kali@kali)~[~/DF_Analysis_Lab]
$ md5sum document.pdf
98c002f7509e6829dc1647e4003ff946  document.pdf

(kali@kali)~[~/DF_Analysis_Lab]
$ md5sum image.jpg
bac137d6fd8aded1d0fa8f1d909ffc24  image.jpg

(kali@kali)~[~/DF_Analysis_Lab]
$ md5sum sample.txt
297dbfcebde5eb9de8db3330d81bdb6c  sample.txt

(kali@kali)~[~/DF_Analysis_Lab]
$ md5sum archive.zip
240b915f94aacf460887452e797e3ce1  archive.zip
```

Observation

The MD5 algorithm generated a unique 128-bit hash value for each file. Even though the files had different formats and sizes, each produced a distinct hash value.

Step 2: Generate SHA256 Hash Values

The SHA256 hashing algorithm was used to generate more secure hash values for each file.

Commands Used

```
sha256sum document.pdf
```

```
sha256sum image.jpg
```

```
sha256sum sample.txt
```

sha256sum archive.zip

Screenshot :

```
(kali㉿kali)-[~/DF_Analysis_Lab]
$ sha256sum document.pdf
fc0e64287aa2cf0a0597c0bc346d3fbd3d3f0631e2a88b1f5dfd19a34d027bf6 document.pdf

(kali㉿kali)-[~/DF_Analysis_Lab]
$ sha256sum image.jpg
97da014453ee8fcf2752662499269adb90eab0b79fa116418231fab9defa342f image.jpg

(kali㉿kali)-[~/DF_Analysis_Lab]
$ sha256sum sample.txt
773a16e897bea467f5c552878a2e885779cbc63cf19ae7b8b54bf02ebd1bf271 sample.txt

(kali㉿kali)-[~/DF_Analysis_Lab]
$ sha256sum archive.zip
37c7b4e5d2b911d0e61923845522adab13ee54893ca11a246ba494f55858c868 archive.zip
```

Observation

SHA256 generated unique 256-bit hash values for all files. Compared to MD5, SHA256 provides stronger security and is widely used for integrity verification.

Step 3: Compare Hashes of Two Identical Files

A duplicate copy of the text file was created to compare hash values.

Command Used

```
cp sample.txt sample_copy.txt
```

Generate Hashes

```
sha256sum sample.txt
```

```
sha256sum sample_copy.txt
```

Screenshot :

```
(kali㉿kali)-[~/DF_Analysis_Lab]
$ cp sample.txt sample_copy.txt

(kali㉿kali)-[~/DF_Analysis_Lab]
$ sha256sum sample.txt
773a16e897bea467f5c552878a2e885779cbc63cf19ae7b8b54bf02ebd1bf271 sample.txt

(kali㉿kali)-[~/DF_Analysis_Lab]
$ sha256sum sample_copy.txt
773a16e897bea467f5c552878a2e885779cbc63cf19ae7b8b54bf02ebd1bf271 sample_copy.txt
```

Result

The hash values of both files were identical because the contents of the files were exactly the same.

Step 4: Modify a File and Generate the Hash Again

The duplicate file was modified by adding new content.

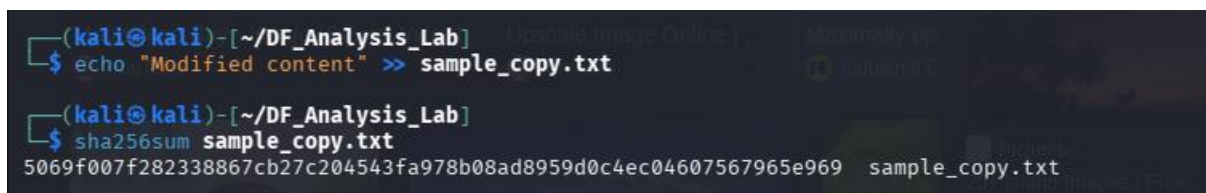
Command Used

```
echo "Modified content" >> sample_copy.txt
```

The SHA256 hash was generated again.

```
sha256sum sample_copy.txt
```

Screenshot :



```
(kali㉿kali)-[~/DF_Analysis_Lab]
$ echo "Modified content" >> sample_copy.txt

(kali㉿kali)-[~/DF_Analysis_Lab]
$ sha256sum sample_copy.txt
5069f007f282338867cb27c204543fa978b08ad8959d0c4ec04607567965e969 sample_copy.txt
```

Result

The hash value changed after modifying the file.

Did the Hash Change?

Yes, the hash value changed.

Why Did the Hash Change?

Hash functions generate a unique digital fingerprint based on the contents of a file. When additional data was added to the file content, resulting in a completely different hash value. This characteristic, known as the **avalanche effect**, ensures that even a small modification produces a significantly different hash. As a result, hashing is widely used in digital forensics to verify file integrity and detect unauthorized changes.

PART 3: METADATA ANALYSIS

Objective

The objective of this task was to analyze file metadata using ExifTool. Metadata provides information about a file such as creation date, author, software used, camera information, and GPS coordinates. This information is useful in digital forensic investigations for identifying the origin and history of files.

Step 1: Install ExifTool

ExifTool was installed to extract metadata from files.

Command Used

`sudo apt update`

`sudo apt install exiftool -y`

Screenshot :

```
└─$ sudo apt update
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.3 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.4 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [104 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [189 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [175 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [891 kB]
Fetched 76.1 MB in 1min 21s (942 kB/s)
878 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```
└─$ sudo apt install exiftool -y
Note, selecting 'libimage-exiftool-perl' instead of 'exiftool'
The following packages were automatically installed and are no longer required:
  libavc1394-0  libipc-run-perl  libpostal1  python3-pluginbase
  libio-pty-perl  libpostal-data  postgresql-common-dev  python3-pysnmp4
Use 'sudo apt autoremove' to remove them.

Upgrading:
  libimage-exiftool-perl  wafw00f

Summary:
  Upgrading: 2, Installing: 0, Removing: 0, Not Upgrading: 876
  809 not fully installed or removed.
  Download size: 0 B / 6,146 kB
  Space needed: 874 kB / 3,766 MB available

Setting up mount (2.42-6)...
Setting up systemd (260.1-1)...
Created symlink '/etc/systemd/system/autovt@.service' → '/usr/lib/systemd/system/getty@.service'.
Setting up cron-daemon-common (3.0pl1-200)...
Setting up libncursesw6:amd64 (6.6+20251231-1+b1)...
Setting up libproc2-0:amd64 (2:4.0.4-9+b2)...
Setting up procps (2:4.0.4-9+b2)...
(Reading database... 428627 files and directories currently installed.)
Preparing to unpack .../wafw00f_2.4.2-1_all.deb...
Unpacking wafw00f (2.4.2-1) over (2.3.2-1)...
Preparing to unpack .../libimage-exiftool-perl_13.55+dfsg-1_all.deb...
Unpacking libimage-exiftool-perl (13.55+dfsg-1) over (13.50+dfsg-1)...
Setting up libip4tc2:amd64 (1.8.13-1)...
Setting up libgoa-1.0-common (3.58.1-1)...
Setting up libxpat1:amd64 (2.8.1-1)...
Setting up bubblewrap (0.11.2-2)...
Setting up libxdot4:amd64 (14.1.2-1+b1)...
Setting up libtext-iconv-perl:amd64 (1.7-9)...
Setting up libgraphite2-3:amd64 (1.3.14-13)...
Setting up liblcms2-2:amd64 (2.17-1.1)...
```

Step 2: Analyze image.jpg Metadata

The metadata of the JPEG image file was examined using ExifTool.

Command Used

```
exiftool image.jpg
```

Screenshot :

```
(kali@kali)-[~/DF_Analysis_Lab]
$ exiftool image.jpg
ExifTool Version Number      : 13.55
File Name                    : image.jpg
Directory                   : .
File Size                    : 414 kB
File Modification Date/Time   : 2026:06:24 17:04:31+05:30
File Access Date/Time        : 2026:06:24 17:07:13+05:30
File Inode Change Date/Time   : 2026:06:24 17:04:31+05:30
File Permissions              : -rw-rw-r--
File Type                    : JPEG
File Type Extension          : jpg
MIME Type                    : image/jpeg
Image Width                  : 1920
Image Height                 : 1080
Encoding Process              : Baseline DCT, Huffman coding
Bits Per Sample              : 8
Color Components              : 3
Y Cb Cr Sub Sampling         : YCbCr4:2:0 (2 2)
Image Size                   : 1920x1080
Megapixels                   : 2.1
```

Observation

Metadata Field	Value
File Name	image.jpg
File Type	JPEG
File Size	414 KB
File Modification Date	2026:06:24 17:04:31+05:30
Image Width	1920 pixels
Image Height	1080 pixels
Image Size	1920 × 1080
Megapixels	2.1 MP
Software Used	Not Available
Camera Information	Not Available
GPS Information	Not Available

Step 3: Analyze document.pdf Metadata

The metadata of the PDF file was examined using ExifTool.

Command Used

```
exiftool document.pdf
```

Screenshot

```
(kali@kali)-[~/DF_Analysis_Lab]
$ exiftool document.pdf
ExifTool Version Number      : 13.55
File Name                    : document.pdf
Directory                    : .
File Size                    : 2.3 MB
File Modification Date/Time   : 2026:06:24 16:54:40+05:30
File Access Date/Time        : 2026:06:24 17:00:11+05:30
File Inode Change Date/Time   : 2026:06:24 16:54:40+05:30
File Permissions              : -rw-rw-r--
File Type                    : PDF
File Type Extension          : pdf
MIME Type                    : application/pdf
PDF Version                   : 1.7
Linearized                   : No
Page Count                   : 5
Producer                     : cairo 1.18.0 (https://cairographics.org)
Creator                      : Mozilla Firefox 140.11.0
Create Date                   : 2026:06:24 16:54:36+05:30
```

Observation

Metadata Field	Value
File Name	document.pdf
File Type	PDF
File Size	2.3 MB
PDF Version	1.7
Linearized	No
Page Count	5
Producer	cairo 1.18.0
Creator	Mozilla Firefox 140.11.0
Create Date	2026:06:24 16:54:36+05:30
Author	Not Available
GPS Information	Not Applicable

Metadata Analysis Summary

Metadata Field	image.jpg	document.pdf
File Type	JPEG	PDF
File Size	414 KB	2.3 MB
Creation Date	Not Available	2026:06:24 16:54:36+05:30
Author	N/A	Not Available
Camera Information	Not Available	N/A
Software Used	Not Available	Mozilla Firefox 140.11.0
GPS Information	Not Available	N/A
Image Dimensions	1920 × 1080	N/A
Page Count	N/A	5

Importance of Metadata in Digital Forensics

Metadata can provide valuable information during an investigation, including:

1. File creation and modification timestamps.
2. Information about the device used to create the file.
3. Software used to generate or edit the file.
4. Geographical location information through GPS data.
5. Evidence regarding file ownership and authenticity.

Observations

1. ExifTool successfully extracted metadata from both files.
2. Different file types contain different categories of metadata.
3. Image files may contain camera and GPS information.
4. PDF files may contain author and software details.
5. Metadata can assist investigators in tracing the origin and history of digital evidence.

Conclusion

Metadata analysis was successfully performed using ExifTool. The extracted metadata provided useful information about the files, including creation details, software information, and other attributes. Metadata analysis is an important component of digital forensics because it helps investigators understand the origin, authenticity, and history of digital evidence.

PART 4: INTEGRITY VERIFICATION

Objective

The objective of this task was to verify the integrity of digital evidence using cryptographic hashing. Hash values were generated before and after modifying a file to demonstrate how even a small change affects the file's hash value.

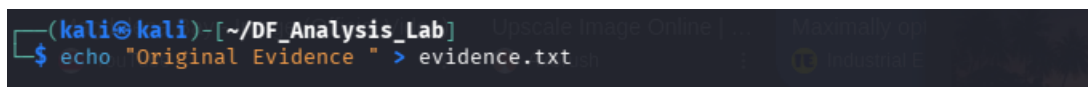
Step 1: Create an Evidence File

A file named **evidence.txt** was created to simulate digital evidence.

Command Used

```
echo "Original Evidence Data" > evidence.txt
```

Screenshot :

A terminal window with a dark background. The prompt is (kali@kali)-[~/DF_Analysis_Lab]. The command entered is echo "Original Evidence Data" > evidence.txt. The output is not visible.

```
(kali@kali)-[~/DF_Analysis_Lab]  
$ echo "Original Evidence Data" > evidence.txt
```

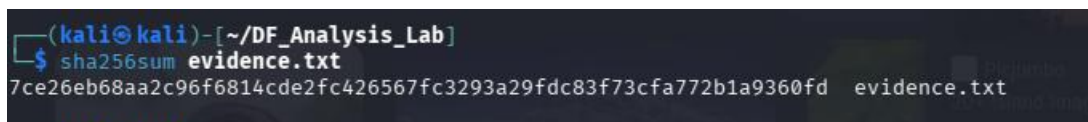
Step 2: Generate the Initial Hash Value

A SHA256 hash was generated for the evidence file to establish its original integrity value.

Command Used

```
sha256sum evidence.txt
```

Screenshot :

A terminal window with a dark background. The prompt is (kali@kali)-[~/DF_Analysis_Lab]. The command entered is sha256sum evidence.txt. The output is 7ce26eb68aa2c96f6814cde2fc426567fc3293a29fdc83f73cfa772b1a9360fd evidence.txt.

```
(kali@kali)-[~/DF_Analysis_Lab]  
$ sha256sum evidence.txt  
7ce26eb68aa2c96f6814cde2fc426567fc3293a29fdc83f73cfa772b1a9360fd evidence.txt
```

Observation

A unique SHA256 hash value was generated for the evidence file. This hash serves as a digital fingerprint of the file.

Step 3: Modify the Evidence File

The evidence file was modified by adding additional content.

Command Used

```
echo "Additional Evidence Data" >> evidence.txt
```

Screenshot :

```
(kali@kali)-[~/DF_Analysis_Lab]
$ echo "Additional Evidence" > evidence.txt
```

Observation

The content of the file was successfully modified by appending new data.

Step 4: Generate the Hash Value Again

After modification, a new SHA256 hash was generated for the evidence file.

Command Used

sha256sum evidence.txt

Screenshot :

```
(kali@kali)-[~/DF_Analysis_Lab]
$ sha256sum evidence.txt
f9af24fc73a7198a77d3ac0d8bbac3e92a4ff8b2afc50ad94a754d5351a38820  evidence.txt
```

Observation

The newly generated hash value was different from the original hash value, indicating that the file contents had changed.

Hash Comparison Table

Stage	SHA256 Hash Value
Before Modification	7ce26eb68aa2c96f6814cde2fc426567fc3293a29fdc83f73cfa772b1a9360fd
After Modification	f9af24fc73a7198a77d3ac0d8bbac3e92a4ff8b2afc50ad94a754d5351a38820

Result

The SHA256 hash value changed after the evidence file was modified.

Did the Hash Change?

Yes, the hash value changed after modification.

Why Did the Hash Change?

The SHA256 algorithm generates a unique hash based on the contents of a file. When the text "Additional Evidence" was added to the file, its contents changed. As a result, a completely new hash value was generated. This demonstrates that even a small modification to a file produces a significantly different hash value.

Observation

- The original SHA256 hash represented the initial state of the evidence file.
- After modifying the file, a different SHA256 hash was generated.
- The change in hash confirmed that the file contents had been altered.
- Hashing is an effective technique for detecting unauthorized changes to digital evidence.

Why Digital Evidence Integrity Is Important

Digital evidence integrity refers to maintaining the original state of digital evidence throughout the investigation process. From the moment evidence is collected until it is examined, stored, or presented in legal proceedings, it must remain unaltered. Any unauthorized modification can compromise the authenticity and credibility of the evidence, potentially affecting the outcome of an investigation. Therefore, preserving evidence integrity is a fundamental requirement in digital forensics.

Significance of Digital Evidence Integrity

1. **Ensures Authenticity**
Verifies that the evidence is genuine and remains in its original form without any unauthorized changes.
2. **Detects Tampering or Alterations**
Helps investigators identify whether a file has been modified, intentionally or unintentionally, after collection.
3. **Maintains the Chain of Custody**
Supports proper documentation and tracking of evidence throughout the investigation process.
4. **Supports Legal Acceptance**
Ensures that digital evidence meets legal standards and can be presented confidently in courts or disciplinary proceedings.
5. **Enhances Investigation Reliability**
Preserves the accuracy and trustworthiness of forensic findings, enabling investigators to draw valid conclusions.

How Hashing Helps Investigators

Hashing is a fundamental technique used in digital forensics to verify the integrity and authenticity of digital evidence. A hash function generates a unique fixed-length value, known as a hash value, based on the contents of a file. This value acts as a digital fingerprint, allowing investigators to determine whether a file has been altered during storage, transfer, or analysis.

Benefits of Hashing in Digital Forensics

- 1. Generates a Unique Digital Fingerprint**
Each file produces a distinct hash value, making it possible to uniquely identify digital evidence.
- 2. Detects Unauthorized Changes**
Any modification to a file, even a minor one, results in a completely different hash value, helping investigators identify tampering.
- 3. Verifies File Authenticity**
Hash values can be compared to confirm that a file is genuine and has not been altered since its collection.
- 4. Ensures Evidence Integrity**
Hashing helps maintain the integrity of digital evidence during storage, transmission, and forensic examination.
- 5. Supports Chain of Custody Procedures**
By recording hash values at different stages of an investigation, investigators can demonstrate that evidence remained unchanged throughout the process.

Security Recommendations

1. Use SHA256 or stronger hashing algorithms for integrity verification.
2. Verify file hashes before and after transferring digital evidence.
3. Remove sensitive metadata before sharing files publicly.
4. Maintain proper chain of custody for digital evidence.
5. Store evidence in secure and access-controlled locations.
6. Regularly verify evidence integrity using cryptographic hashes.
7. Keep forensic tools updated to ensure accurate analysis.